SQL Assignment: LLM Agent Platform Database

Create a relational database for a platform where users build AI agents on top of large language models (LLMs). The goal is to practice creating tables, relationships, primary keys, foreign keys, a many-to-many link table, constraints and SQL queries with joins, grouping and calculations.

> **Background.** An *LLM agent* is a program that uses a language model to work on a task step by step. Each agent runs on one **model** (for example a large or a small model from some provider) and is allowed to use some **tools**: web search, running code, querying a database, calling an API, reading files. Each time a user gives an agent a task, that is a **run**. During a run the agent may make several **tool calls**. Models charge money per token (a token is a small piece of text), so the platform records how many tokens each run used.

1. Database Tables

users

Column	Meaning
`user_id`	Unique number of the user
`first_name, last_name`	User's name
`email`	User's email
`registration_date`	When the user signed up

models

Column	Meaning
`model_id`	Unique number of the model
`model_name`	Name of the model
`provider`	Company that offers the model
`context_window`	Maximum number of tokens the model can read at once
`input_price`	Price in USD per 1 million input tokens
`output_price`	Price in USD per 1 million output tokens

agents

Column	Meaning
`agent_id`	Unique number of the agent
`user_id`	The user who owns the agent
`model_id`	The model the agent runs on
`agent_name`	Name of the agent
`system_prompt`	Instructions that tell the agent how to behave
`temperature`	Creativity setting of the model (0 = strict, 2 = very creative)
`created_date`	When the agent was created

tools

Column	Meaning
`tool_id`	Unique number of the tool
`tool_name`	Name of the tool
`description`	What the tool does
`tool_type`	Kind of tool

agent_tools

Column	Meaning
`agent_id`	The agent
`tool_id`	A tool this agent is allowed to use

runs

Column	Meaning
`run_id`	Unique number of the run
`agent_id`	The agent that did the run
`task_description`	The task the user gave the agent
`start_time, end_time`	When the run started and finished (date and time)
`status`	State of the run
`input_tokens, output_tokens`	How many tokens the model read and wrote

tool_calls

Column	Meaning
`call_id`	Unique number of the tool call
`run_id`	The run during which the call happened
`tool_id`	The tool that was called
`call_time`	When the call happened
`call_input`	What the agent sent to the tool
`duration_ms`	How long the call took, in milliseconds
`call_status`	Result of the call

2. Relationships

- A user can own many agents. Each agent belongs to one user.
- A model can be used by many agents. Each agent runs on one model.
- An agent can use many tools, and a tool can be used by many agents (many-to-many through `agent_tools`).
- An agent can have many runs. Each run belongs to one agent.
- A run can have many tool calls. Each tool call belongs to one run.
- A tool can be called many times. Each tool call is for one tool.
- The relationships between tables must be implemented using foreign keys.

3. Required Constraints

- Every table must have a PRIMARY KEY. In `agent_tools` the primary key is the pair (agent_id, tool_id), so the same tool cannot be given to the same agent twice.
- All foreign keys must reference the appropriate tables, and foreign key columns must be NOT NULL.
- User email must be UNIQUE and NOT NULL.
- Model name must be UNIQUE and NOT NULL. Provider must be NOT NULL.
- Tool name must be UNIQUE and NOT NULL.
- Agent name and run task description must be NOT NULL. Run start time must be NOT NULL.
- Context window must be greater than 0. Model prices must be greater than or equal to 0.
- Temperature must be between 0 and 2.
- Tool type must be one of: `'SEARCH'`, `'CODE'`, `'DATABASE'`, `'API'`, `'FILE'`.
- Run status must be one of: `'RUNNING'`, `'COMPLETED'`, `'FAILED'`, `'CANCELLED'`.
- Input and output tokens must be greater than or equal to 0.
- A run's end time cannot be earlier than its start time.
- A run with status `'RUNNING'` must have no end time; every other status must have an end time.
- Tool call duration must be greater than or equal to 0.
- Tool call status must be one of: `'SUCCESS'`, `'ERROR'`, `'TIMEOUT'`.

4. Sample Data

- 5 users
- 4 models
- 6 agents
- 6 tools
- 12 agent–tool assignments
- 10 runs
- 15 tool calls

The data must allow every query below to return a meaningful result. In particular:

- at least one agent that has never been run, and at least one tool that has never been called;
- at least one run of each status (running, completed, failed, cancelled);
- at least one tool call with `'ERROR'` and one with `'TIMEOUT'`;
- agents may only call tools that are assigned to them in `agent_tools`.

5. Required SQL Queries

1. Display all users.
2. Display all models with their provider and prices.
3. Display all agents with the owner's name and the model name.
4. Display all tools available to a selected agent.
5. Display all runs of a selected agent, ordered by start time.
6. Count how many runs each agent has (agents with no runs must show 0).
7. Display all failed runs with the agent name.
8. Display models with a context window larger than 100 000 tokens.
9. Calculate the cost of each run in USD:
   `(input_tokens × input_price + output_tokens × output_price) / 1 000 000`
10. Display the total number of tokens and the total cost for each user (users with no runs must show 0).
11. For each tool, show how many times it was called, how many calls ended with an error or a timeout, and the average duration (tools that were never called must show 0).
12. Display all tool calls with their run and agent details.
13. Display the duration of each finished run in seconds, longest first.
14. Display agents that have never been run.
15. Data check: find tool calls where the agent used a tool that is not assigned to it (the result should be empty).

6. What to Submit

- One `.sql` file that creates the tables, adds the constraints, inserts the sample data and contains all the queries. It must run from top to bottom without errors.
- An ER diagram of the database (for example made in drawSQL or dbdiagram.io).